

openGauss中视图、索引和事务的应用

难度（最高三星）：★★★ 建议学时（2-4学时）：4学时

实验说明

学习地图

本项目实验案例集是实现一个学生成绩管理系统，共分为9个实验任务，按照难度等级分为初级、中级、高级，如图1所示。



图1 学生成绩管理系统实验

本次实验主要完成其中的任务4，主要实现学生管理系统中视图、索引以及事务的使用。

实验环境准备

- (1) openGauss的安装与部署：该部分请参照实验任务1【[openGauss表空间、模式、权限管理](#)】进行安装与部署。

(2) 数据库创建：请参照实验任务1【[openGauss表空间、模式、权限管理](#)】，或[直接下载相关SQL语句](#)进行数据库表空间创建、数据库模式创建、用户权限管理。

(3) 学生成绩管理系统的数据表以及相关约束：请参照实验任务2【[openGauss数据表创建与数据表约束](#)】，或[直接下载相关SQL语句](#)并进行运行生成相关的表。

(4) 对学生成绩管理系统中的表数据进行插入：请参照实验任务3【[openGauss数据库SQL语法](#)】，或[直接下载相关SQL语句](#)并进行运行生成相关的表。

实验任务

任务描述

本次实验共计六个子任务，主要是完成对学生成绩管理系统的数据表使用视图、索引、事务技术。整个实验会有详细的需求说明、实施步骤和关键代码。

学习目标

完成本任务的学习后，你应当能：

1. 学会使用可视化方式创建视图获取多表中数据；
2. 学会创建并使用索引；
3. 学会使用事务。

任务准备

前置知识

本实验案例，需提前学习openGauss的基本概念、作用及基础操作。可通过如下途径进行

学习：

- 数字教材：《[数据库原理与应用：基于openGauss](#)》
- 技术文档：[openGauss白皮书](#)

任务实施

任务1：使用Data Studio可视化创建视图查看学生详细信息

本实验的具体操作可参见视频。

训练要点：使用可视化方式创建视图获取多表中数据。

需求说明：需要从学生表（student）和年级表（grade）两个表中获取学生的相关信息。

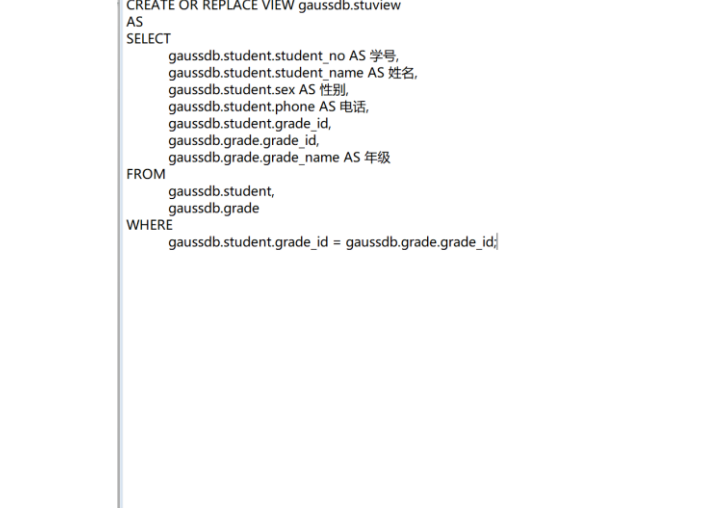
关键步骤如下：

步骤一：在数据库中选择视图并右键点击创建，如图2所示。



图2 创建视图1

步骤二：在展开的界面中录入如图3所示的数据。



The screenshot shows a database editor window titled "编辑视图" (Edit View). The window has two tabs: "编辑视图" (selected) and "预览" (Preview). The SQL code displayed is as follows:

```
CREATE OR REPLACE VIEW gaussdb.stuvview
AS
SELECT
    gaussdb.student.student_no AS 学号,
    gaussdb.student.student_name AS 姓名,
    gaussdb.student.sex AS 性别,
    gaussdb.student.phone AS 电话,
    gaussdb.student.grade_id,
    gaussdb.grade.grade_id,
    gaussdb.grade.grade_name AS 年级
FROM
    gaussdb.student,
    gaussdb.grade
WHERE
    gaussdb.student.grade_id = gaussdb.grade.grade_id;
```

At the bottom of the window, there are two buttons: "上一步" (Previous Step) and "完成" (Finish).

图4 创建视图3

2025-9-4

```
CREATE OR REPLACE VIEW gaussdb.stuvview
AS
SELECT
    gaussdb.student.student_no AS 学号,
    gaussdb.student.student_name AS 姓名,
    gaussdb.student.sex AS 性别,
    gaussdb.student.phone AS 电话,
    gaussdb.grade.grade_name AS 年级
FROM
    gaussdb.student,
    gaussdb.grade
WHERE
    gaussdb.student.grade_id = gaussdb.grade.grade_id;
```

运行结果如图5所示。

	学号	姓名	性别	电话	年级
1	10000	Alice	女	13900000001	大一年级
2	10001	Bob	男	13900000002	大一年级
3	10002	Charlie	女	13900000003	大一年级
4	10011	David	男	13900000004	大一年级
5	10012	Emma	女	13900000005	大一年级
6	20011	Frank	男	13900000006	大二年级
7	20012	Grace	女	13900000007	大二年级
8	30015	Hannah	女	13900000008	大三年级
9	30021	Isabella	女	13900000009	大三年级
10	40023	Jacob	男	13900000010	大四年级

图5 创建视图4

总结：

Data Studio可以使创建视图更便捷，实现了多表的连接，并且通过更改自动生成的SQL语句，能够实现更多的功能。

任务2：创建并使用视图查看学生考试时间以及考试成绩。

本实验的具体操作可参见视频。

训练要点：使用视图获取多表中数据。

需求说明：这里需要使用四张表（学生表、年级表、课程表、成绩表），分别罗列出每个学生的课程考核时间（最新一次）和考核成绩。

实现思路 and 关键代码：

(1) 创建视图，本次练习使用SQL语句完成，主要使用了第四章中的SQL连接查询等语句。

```
CREATE VIEW student_result1
AS
SELECT Student_Name AS 姓名, Student.student_no AS 学号, Student_Result AS 成绩,
       Subject_Name AS 课程名称, Exam_Date AS 考试日期
FROM Student
INNER JOIN Result ON Student.Student_No = Result.Student_No
INNER JOIN Subject ON Result.Subject_No = Subject.Subject_No
WHERE Subject.Subject_No = (
    SELECT Subject_No FROM Subject WHERE Subject_Name = '数据库高级特性')
AND Exam_Date = (
    SELECT MAX(Exam_Date) FROM Result, Subject
    WHERE Result.Subject_No = Subject.Subject_no AND subject_name = '数据库高级特性')
```

(2) 编码查看视图的运行结果，如图6所示。

SELECT * FROM vw_student_result_all;					
	姓名	学号	成绩	课程名称	考试日期
1	Hannah	30015	60	数据库高级特性	2024-02-15 00:00:00
2	Isabella	30021	23	数据库高级特性	2024-02-15 00:00:00

图6 运行结果

任务3：创建视图，得到大一年级学生平均成绩排行

本实验的具体操作可参见视频。

训练要点：使用视图获取多表中数据。

需求说明：统计每个学生大一所有课程的平均成绩并进行排序。

实现思路 and 关键代码：

(1) 创建视图，实现查询学生大一参加考试的平均成绩，每门课的成绩以该生参加最后一次考试为准。

```
CREATE VIEW vw_student_result_all AS
SELECT
    s.student_name AS "姓名",
    s.student_no AS "学号",
    s.phone AS "联系电话",
    t.grade_name AS "学期",
    t.avg_score AS "平均成绩"
FROM student s
LEFT JOIN (
    SELECT
        r.student_no,
        g.grade_name,
        AVG(r.student_result) AS avg_score
    FROM result r
    INNER JOIN (
        SELECT
            student_no,
            subject_no,
            MAX(exam_date) AS exam_date
        FROM result
```

```

GROUP BY student_no, subject_no
) t1 ON r.exam_date = t1.exam_date
AND r.subject_no = t1.subject_no
AND r.student_no = t1.student_no
INNER JOIN subject sub ON sub.subject_no = r.subject_no
INNER JOIN grade g ON g.grade_id = sub.grade_id
GROUP BY r.student_no, g.grade_name
) t ON s.student_no = t.student_no
WHERE t.grade_name='大一年级';

```

(2) 编码查看视图的运行结果，并按照成绩由高到低排序。

```
SELECT * FROM vw_student_result_all ORDER BY "平均成绩" DESC;
```

运行结果如图7所示。

	姓名	学号	联系电话	学期	平均成绩
1	David	10011	13900000004	大一年级	83.5000000000000000
2	Charlie	10002	13900000003	大一年级	83.0000000000000000
3	Bob	10001	13900000002	大一年级	76.0000000000000000
4	Emma	10012	13900000005	大一年级	76.0000000000000000
5	Alice	10000	13900000001	大一年级	60.0000000000000000

图7 获得学生大一考试的平均成绩

任务4：创建并使用索引查询学生出生日期

本实验子任务4&5的具体操作可参见视频。

训练要点：向表中循环插入数据，创建并使用索引。

需求说明：利用一个循环来插入1000条学生数据，要求如下：

- (1) 为了区别原有的学号，新的学号为6位，学号第一位为1，依次递增。
- (2) 名字为stu1-1000。

- (3) 性别随机，只能为“男”或“女”。
- (4) 手机号开头为139，后面8位数字按顺序递增。
- (5) 地址不填。
- (6) 生日随机在1980年1月1日之后。
- (7) 邮箱为stu1-1000@123.com。
- (8) 身份证号为18位数字，开头为“110000”依次递增。

插入数据后给出生日期建立索引，查询出生日期在2003年1月1日到2003年6月30日的所有记录。要求输出：学生姓名和出生日期，并比较在没有索引和有索引的情况下输出的时间。

实现思路 and 关键代码：

- (1) 插入1000条学生数据

```
DO $$
DECLARE
    v_student_no INT;
    v_student_name VARCHAR(50);
    v_sex VARCHAR(3);
    v_grade_id INT;
    v_phone VARCHAR(11);
    v_address VARCHAR(255) := '';
    v_born_date TIMESTAMP;
    v_email VARCHAR(50);
    v_identity_card VARCHAR(18);
    v_count INT := 1;
    v_exist_count INT;
BEGIN
    WHILE v_count <= 1000 LOOP
        v_student_no := 100000 + v_count;
        v_student_name := 'stu' || LPAD(CAST(v_count AS VARCHAR(4)), 4, '0');
```

```
v_sex := CASE FLOOR(RANDOM() * 2)
          WHEN 0 THEN '男'
          WHEN 1 THEN '女'
        END;
v_grade_id := v_student_no / 100000;
v_phone := '139' || LPAD(CAST((v_count + 10) AS VARCHAR(8)), 8, '0');
v_born_date := '1980-01-01'::TIMESTAMP + INTERVAL '1 day' * FLOOR(RANDOM()
* (16000));
v_email := 'stu' || LPAD(CAST(v_count AS VARCHAR(4)), 4, '0') || '@123.com';
v_identity_card := '110000' || LPAD(CAST((v_count + 10) AS VARCHAR(12)), 12, '0');

INSERT INTO Student VALUES(v_student_no, v_student_name, v_sex, v_grade_id,
v_phone, v_address, v_born_date, v_email, v_identity_card);

v_count := v_count + 1;
END LOOP;
END;
$$;
```

(2) 进行无索引查询，编码输出学生姓名和出生日期

```
SELECT
    student_name AS 学生姓名,
    born_date AS 出生日期
FROM
    Student
WHERE
    born_date BETWEEN '2003-01-01' AND '2003-06-30';
```

运行结果如图8所示。

	学生姓名	出生日期
1	stu0030	2003-05-18 00:00:00
2	stu0211	2003-04-17 00:00:00
3	stu0314	2003-02-07 00:00:00
4	stu0406	2003-03-23 00:00:00
5	stu0514	2003-04-10 00:00:00
6	stu0842	2003-05-21 00:00:00
7	stu0953	2003-06-08 00:00:00

图8 学生姓名和出生日期

没有索引情况下输出的时间（获取输出时间需要在SELECT语句前加上EXPLAIN

ANALYZE)

```
EXPLAIN ANALYZE SELECT
  student_name AS 学生姓名,
  born_date AS 出生日期
FROM
  Student
WHERE
  born_date BETWEEN '2003-01-01' AND '2003-06-30';
```

无索引查询时间如图9所示。

QUERY PLAN	
1	Seq Scan on student (cost=0.00..30.15 rows=12 width=15) (actual time=0.016..0.126 rows=10 loops=1)
2	Filter: ((born_date >= '2003-01-01 00:00:00'::timestamp without time zone) AND (born_date <= '2003-06-30 00:00:00'::timestamp without time zone))
3	Rows Removed by Filter: 1000
4	Total runtime: 0.199 ms

图9 无索引查询时间

(3) 创建出生日期的索引

```
CREATE INDEX IX_Student_Borndate ON student(born_date);
```

有索引查询时间如图10所示。

QUERY PLAN	
1	Bitmap Heap Scan on student (cost=4.37..19.64 rows=12 width=15) (actual time=0.096..0.105 rows=10 loops=1)
2	Recheck Cond: ((born_date >= '2003-01-01 00:00:00'::timestamp without time zone) AND (born_date <= '2003-06-30 00:00:00'::timestamp without time zone))
3	Heap Blocks: exact=8
4	-> Bitmap Index Scan on ix_student_borndate (cost=0.00..4.37 rows=12 width=0) (actual time=0.089..0.089 rows=10 loops=1)
5	Index Cond: ((born_date >= '2003-01-01 00:00:00'::timestamp without time zone) AND (born_date <= '2003-06-30 00:00:00'::timestamp without time zone))
6	Total runtime: 0.185 ms

图10 有索引查询时间

上述时间差别不是很大，主要原因是数据量少。

任务5：创建并使用索引优化查询教师信息

训练要点：向表中循环插入数据，创建并使用索引。

需求说明：

- (1) 创建一张教师年龄表，列有教师编号、教师姓名和年龄。
- (2) 插入1000万条数据，教师年龄在25岁到65岁。
- (3) 建立教师年龄的索引。
- (4) 分析在有无索引的时候查询45岁教师的教师信息所需的时间。

实现思路 and 关键代码：

- (1) 创建教师表并插入数据

```
CREATE TABLE teacher (  
    teacher_no SERIAL PRIMARY KEY,  
    teacher_name VARCHAR(50),  
    age INT  
);  
  
DO $$  
DECLARE  
    i INT := 1;  
BEGIN  
    WHILE i <= 10000000 LOOP --1000万条数据  
        INSERT INTO teacher (teacher_name, age)  
        VALUES (  
            'teacher' || i,  
            floor(random() * 100)  
        );  
        i := i + 1;  
    END LOOP;  
END;
```

```
);
i := i + 1;
END LOOP;
END;
$;
```

没有索引的情况下查询45岁教师的教师信息如图11所示。

EXPLAIN ANALYZE SELECT * FROM teacher WHERE age = 45;	
QUERY PLAN	
1	Seq Scan on teacher (cost=0.00..144904.44 rows=32321 width=51) (actual time=0.019..1614.866 rows=100293 loops=1)
2	Filter: (age = 45)
3	Rows Removed by Filter: 9899707
4	Total runtime: 1620.563 ms

图11 没有索引的情况下查询45岁教师的教师信息

(2) 创建教师年龄的索引

```
CREATE INDEX IX_Teacher_Age ON teacher (age);
```

(3) 比较在没有索引和有索引的情况下输出的时间（获取输出时间需要在SELECT语句前加上EXPLAIN ANALYZE)

有索引的情况下查询45岁教师的教师信息如图12所示。

EXPLAIN ANALYZE SELECT * FROM teacher WHERE age = 45;	
QUERY PLAN	
1	Bitmap Heap Scan on teacher (cost=939.75..64612.26 rows=50000 width=51) (actual time=19.140..91.165 rows=100293 loops=1)
2	Recheck Cond: (age = 45)
3	Heap Blocks: exact=50874
4	-> Bitmap Index Scan on ix_teacher_age (cost=0.00..927.25 rows=50000 width=0) (actual time=11.317..11.317 rows=100293 loops=1)
5	Index Cond: (age = 45)
6	Total runtime: 97.066 ms

图12 有索引的情况下查询45岁教师的教师信息

通过练习可以看到，一旦数据达到了一定数量，使用索引明显加快了查询速度，这也是使用索引的好处。

任务6：通过事务实现银行转账。

本实验的具体操作可参见视频。

训练要点：事务在数据操作中的应用。

需求说明：银行的转账交易，从一个账户转账到另一个账户，要求保存交易信息。

实现思路 and 关键代码：

(1) 创建账户表和添加约束和添加数据。

```
-- 创建账户表
CREATE TABLE Account (
    account_id serial PRIMARY KEY, -- 账户ID, 主键, 自增
    account_name VARCHAR(255) NOT NULL, -- 账户名称
    balance DECIMAL(15, 2) NOT NULL -- 账户余额, 保留两位小数
);

-- 添加余额不小于0的条件约束
ALTER TABLE Account
ADD CONSTRAINT balance_non_negative CHECK (balance >= 0);

-- 插入示例账户
INSERT INTO Account (account_name, balance) VALUES ('Alice', 300.00);
INSERT INTO Account (account_name, balance) VALUES ('Bob', 100.00);
```

(2) 启动事务，进行转账。

```
-- 事务1
-- 启动事务
BEGIN;

-- 从源账户扣除转账金额
UPDATE Account
SET balance = balance - 200
WHERE account_id = '1';
```

```
-- 将转账金额添加到目标账户
UPDATE Account
SET balance = balance + 200
WHERE account_id = '2';
--如果运行没错也可以回滚事务
ROLLBACK;
```

将200元从id为1的账户转移到id为2的账户能够正常进行，但执行了ROLLBACK之后，上述转账的业务撤销了。也就是说，一旦执行了ROLLBACK，无论上面的流程是否执行成功，都进行了回滚，回到初始状态。

下面事务2中，使用了COMMIT，这就保证上面的流程操作不是都成功，就是都失败。

```
--事务2
-- 启动事务
BEGIN;
-- 从源账户扣除转账金额
UPDATE Account
SET balance = balance - 200
WHERE account_id = '1';
-- 将转账金额添加到目标账户
UPDATE Account
SET balance = balance + 200
WHERE account_id = '2';
-- 提交事务
COMMIT;
```

事务3中，由于从账户1中转出了200，导致账户1的余额出现了负数，导致违反了表的约束，运行出错，执行了回滚。

```
--事务3
-- 启动事务
BEGIN;
-- 从源账户扣除转账金额（出现余额不足的错误，无法继续进行操作，提示回滚）
UPDATE Account
SET balance = balance - 200
WHERE account_id = '1';
-- 将转账金额添加到目标账户
UPDATE Account
SET balance = balance + 200
WHERE account_id = '2';
--运行出错回滚事务
ROLLBACK;
```

随堂测试

1. ()包含了一组数据库操作命令，并且所有的命令作为一个整体一起向系统提交或撤销操作请求

A. 事务

B. 更新

C. 插入

D. 以上都不是
2. 对数据库的修改必须遵循的规则是：要么全部完成，要么全不修改。这点可以认为是事务的()特性

- A. 一致的
 - B. 持久的
 - C. 原子的
 - D. 隔离的
3. 当一个事务提交或回滚时，数据库中的数据必须保持在()状态
- A. 隔离的
 - B. 原子的
 - C. 一致的
 - D. 持久的
4. 下列的()语句用于清除自最近的事务语句以来所有的修改
- A. COMMIT TRANSACTION
 - B. ROLLBACK TRANSACTION
 - C. BEGIN TRANSACTION
 - D. SAVE TRANSACTION
5. 下列关于视图的说法，错误的是()
- A. 可以使视图集中数据、简化和定制不同用户对数据库的不同要求
 - B. 视图可以使用户只关心他感兴趣的某些特定数据和他们所负责的特定任务
 - C. 视图可以让不同的用户以不同的方式看到不同或者相同的数据集



D. 视图不能用于连接多表